VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

SEMINAR REPORT

CS163

Group members:

1. 25125052 - Võ Thành Đạt
2. 25125055 - Phạm Huy Khánh
3. 25125068 - Lê Phi Truyền
4. 25125089 - Lê Trọng Đại Nghĩa

19th April 2026

Contents

1 Introduction **2**

1.1 Overview of Quadtrees 2

1.2 Historical Background 2

2 Notation **2**

2.1 Implementation: 2

2.2 Time Complexity: 3

2.3 Space Complexity: 3

3 Range Queries Problem **4**

3.1 Problem Statement 4

3.2 Algorithm 4

3.3 Performance Comparison: Brute Force vs. Quadtree 5

3.3.1 Time Performance 5

3.3.2 Space Overhead 6

4 Programming Assignments **7**

4.1 Logical OR of Two Quad-Trees (LeetCode 558) 7

4.2 Construct Quad Tree (LeetCode 427) 9

4.3 Retrieve Cell Value in a Quad-Tree 11

4.4 Counting Necessary Nodes (Codeforces 2074F) 13

5 Appendices **16**

1 Introduction

1.1 Overview of Quadtrees

A **Quadtree** is a tree data structure in which each internal node has exactly four children. It is most often used to partition a two-dimensional space by recursively subdividing it into four quadrants or regions. The data represented may be points, regions, lines, or curves. Quadtrees are fundamental in fields that require spatial indexing, such as computer graphics, geographic information systems (GIS), and image processing. By using a hierarchical representation, Quadtrees allow for efficient spatial queries, reducing the time complexity of operations like range searching or collision detection from $O(N)$ to $O(\log N)$.

1.2 Historical Background

The term "Quadtree" was first coined by **Raphael Finkel** and **J.L. Bentley** in **1974** to describe a data structure for retrieving points based on their coordinates in a 2D plane. However, the concept of recursive decomposition of space has older roots in mathematics and cartography.
Shortly after its introduction, the structure was adapted into the **Region Quadtree** by **Hunter** in **1978** for image processing and computer graphics. During the 1980s and 1990s, new variants such as the *Point Quadtree*, *MX Quadtree*, and *PR Quadtree* were developed for different data types. Today, Quadtrees power many practical systems, including map tiling and collision detection.

2 Notation

2.1 Implementation:

- A Quad-Tree is implemented as a hierarchical tree structure where each internal node has exactly four children corresponding to four spatial quadrants.
- Each node stores information about a specific region of the space. The node may either be a *leaf node*, representing a uniform region, or an *internal node* that subdivides the region into four smaller subregions.
- The tree is typically constructed using a recursive divide-and-conquer strategy. If a region

satisfies a stopping condition (for example, uniform data or minimum region size), a leaf node is created. Otherwise, the region is subdivided into four equal quadrants.

- Each recursive step creates four child nodes representing the top-left, top-right, bottom-left, and bottom-right subregions.
- Quad-Trees are commonly used for spatial partitioning problems such as image compression, collision detection, and geographic information systems.

2.2 Time Complexity:

- The time complexity is $\mathcal{O}(n^2 \log n)$ where n is the side length of the grid. The analysis breaks down follows:
- In the worst case, the Quad-Tree needs to be fully constructed when no regions can be merged into leaves.
- The recursion depth is $\log_2 n$ (since we divide the grid into quarters at each level).
- At each level k of the recursion tree, there are 4^k subproblems.
- Each subproblem at level k processes a grid of size $(n/2^k) \times (n/2^k) = \frac{n^2}{4^k}$ elements.
- Total work at level k: $4^k \times \frac{n^2}{4^k} = n^2$.
- Since there are $\log_2 n$ levels, the total time complexity is $\mathcal{O}(n^2 \log n)$.

2.3 Space Complexity:

- The space complexity is $\mathcal{O}(n^2)$. The analysis breaks down follows:
- **Recursion stack:** $\mathcal{O}(\log n)$ — The maximum depth of recursion is $\log_2 n$ when dividing the grid into quarters.
- **Quad-Tree nodes:** $\mathcal{O}(n^2)$ — In the worst case where no merging is possible, we create $\mathcal{O}(n^2)$ nodes (approximately $\frac{4^{\log n}-1}{3} = \frac{n^2-1}{3}$ internal nodes plus leaf nodes).
- The dominant factor is the space for storing the Quad-Tree nodes, resulting in $\mathcal{O}(n^2)$ total space complexity.

3 Range Queries Problem

3.1 Problem Statement

Given a dataset of N points (x_i, y_i) in a 2-dimensional space. A query defines an axis-aligned rectangle specified by its top-left corner (x_1, y_1) and bottom-right corner (x_2, y_2).
The objective is to efficiently count the total number of points that fall within the boundaries of each query rectangle. While a naive brute-force approach requires $O(N)$ time per query by iterating through every point, scaling to large datasets requires a spatial partitioning data structure to reduce the search space.

3.2 Algorithm

To optimize the search process, we implement a Point-Region (PR) Quadtree. Each node manages a specific spatial bounding box. To enhance search efficiency and memory usage, actual point coordinates are stored exclusively in the leaf nodes.

- **Tree Construction (Insertion):** Starting from the root node, we determine which quadrant a new point belongs to and traverse down to the corresponding leaf node. If inserting the point causes the leaf node to exceed its predefined capacity, the node is subdivided into 4 child nodes, and the existing points are redistributed.
  Complexity: In the average case with a balanced tree, a single insertion takes $O(\log N)$. The total time to build the tree for N points is $O(N \log N)$.

- **Query Evaluation (Range Counting):** To count the points within a query rectangle R, we traverse the tree using a recursive Divide and Conquer approach. At each node, we evaluate the spatial relationship between the node's Bounding Box (BB) and the query rectangle R:

  1. **Fully Outside (Pruning):** If the node's BB does not intersect R at all, there are no valid points in this branch. We immediately return 0. This spatial pruning is the primary source of the algorithm's speedup.

  2. **Fully Inside (Aggregation):** If the node's BB is completely enclosed by R, every point within this node (and its sub-tree) is guaranteed to be a valid answer. We return the pre-calculated number of points in this node without traversing deeper.

3. **Intersecting (Partial Overlap):** If the node's BB overlaps the boundary of R, we cannot make a definitive assumption. We must recursively call the query function on its 4 child nodes and sum their returning results.

- **Theoretical Time Complexity:**

  Average Case: $O(\sqrt{N})$. The performance bottleneck is the number of nodes that fall into the "Intersecting" case. In a balanced PR Quadtree representing a uniform distribution, any 1D boundary line of the query rectangle will intersect at most $O(\sqrt{N})$ regions.

  Worst Case: $O(N)$. If the dataset is highly concentrated on the boundaries of the query rectangle, or if the tree is entirely unbalanced, the algorithm cannot prune effectively and degrades to checking nearly all points.

3.3 Performance Comparison: Brute Force vs. Quadtree

3.3.1 Time Performance

By analyzing the **Total Query Time** at a small scale ($N = 10,000$) versus a large scale ($N = 1,000,000$), we observe a massive divergence in algorithmic efficiency.

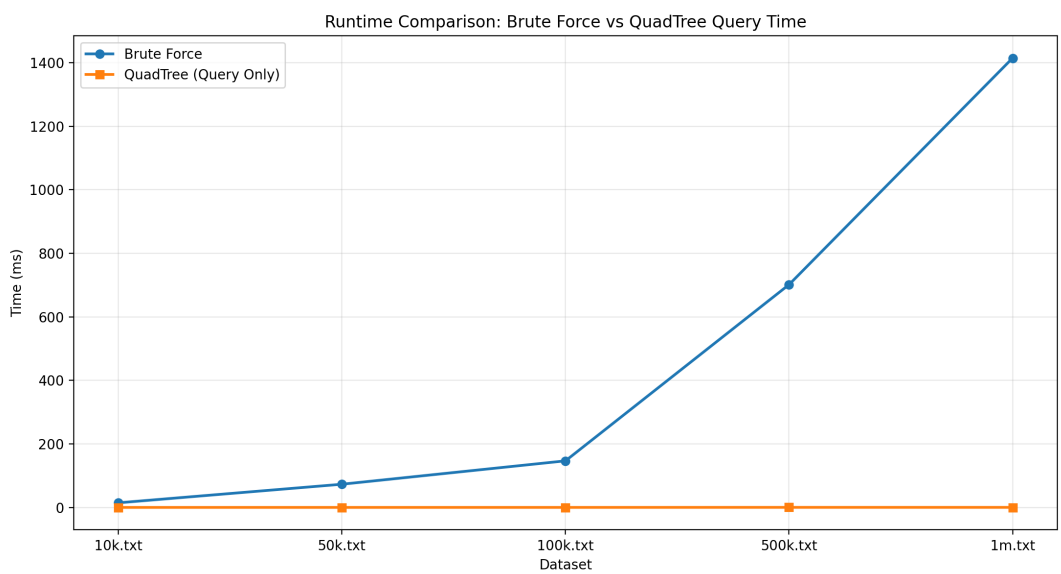

Figure 1: Query Time Complexity Comparison Chart

- **Brute Force ($O(N)$):** The data reflects strict linear scaling. As N increases 100-fold (from 10k to 1m), the query time scales proportionally, jumping from ~14.5ms to over 1.41 seconds.

Data Set Size (N)	Brute Force Time	Quadtree Time	Speedup Factor
10k	14.537 ms	0.010 ms	∼1,453×
50k	73.026 ms	0.023 ms	∼3,175×
100k	146.629 ms	0.036 ms	∼4,073×
500k	701.384 ms	0.181 ms	∼3,875×
1m (1 million)	1414.460 ms	0.140 ms	∼10,103×

Table 1: Comparison of execution times and algorithmic speedup factors.

- **Quadtree ($O(\sqrt{N})$):** Due to sub-linear scaling and aggressive spatial pruning, the execution time remains incredibly low. Scaling the dataset 100-fold only adds fractions of a millisecond to the query, resulting in an extraordinary ∼10,103× speedup at the 1 million point mark.

3.3.2 Space Overhead

The following table illustrates the classic Space-Time Trade-off, showcasing the memory "tax" paid to achieve the Quadtree's massive speedups.

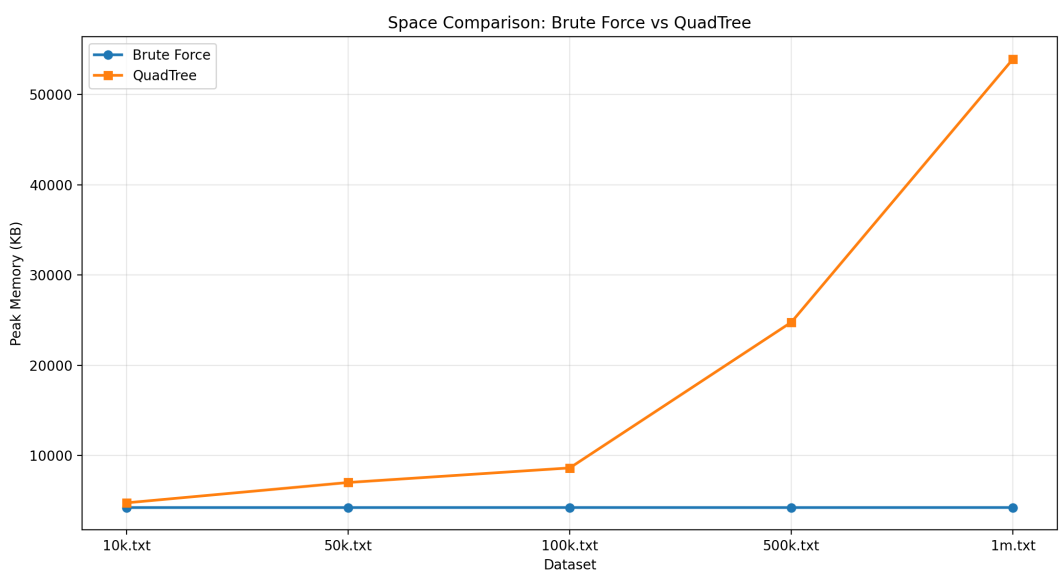

Figure 2: Space Complexity Comparison Chart (RAM Usage)

- **Brute Force:** Highly memory-efficient. The RAM footprint remains virtually constant at ∼4.21 MB, as it only requires storing the raw contiguous array of coordinate data.
- **Quadtree:** Requires significantly more memory allocation. At $N = 1,000,000$, RAM usage

Data Set Size (N)	Brute Force RAM	Quadtree RAM
10k	∼4.21 MB	∼4.74 MB
50k	∼4.21 MB	∼6.99 MB
100k	∼4.22 MB	∼8.61 MB
500k	∼4.21 MB	∼24.77 MB
1m (1 million)	∼4.22 MB	∼53.96 MB

Table 2: Memory footprint comparison between data structures.

expands to ∼53.96 MB to accommodate the overhead of the tree structure, including internal nodes, pointers, and bounding box coordinates.

4 Programming Assignments

4.1 Logical OR of Two Quad-Trees (LeetCode 558)

Problem Description

This problem asks us to merge two $n \times n$ binary matrices represented by Quad-Trees using a logical OR operation.

A Quad-Tree is a tree data structure where each internal node has exactly four children representing the four quadrants: top-left, top-right, bottom-left, and bottom-right. Each node in the tree has two properties:

- `val`: A boolean value (True for 1s, False for 0s)
- `isLeaf`: A boolean indicating whether the node is a leaf (True) or has four children (False)

The construction follows these rules:

- **If a region contains all the same values** (all 0s or all 1s):
  - Create a leaf node with isLeaf = True
  - Set val to the grid's value (True for 1, False for 0)
  - The four children are set to null

- **If a region contains mixed values** (both 0s and 1s):
  - Create an internal node with isLeaf = False
  - Set val to any value (it doesn't matter since it's not a leaf)
  - Divide the current region into four equal quadrants
  - Recursively construct four children nodes for each quadrant

Solution Approach

This solution using recursion to construct the Quad-Tree. Here is how the solution works step by step.

Step 1: Base cases

```
if (!quadTree1 && !quadTree2) return nullptr;
if (!quadTree1) return quadTree2;
if (!quadTree2) return quadTree1;
if (quadTree1->isLeaf && quadTree1->val) return quadTree1;
if (quadTree2->isLeaf && quadTree2->val) return quadTree2;
if (quadTree1->isLeaf && quadTree2->isLeaf) return quadTree1;
```

- **Null checks:** If both tree is null, return null pointer. If either tree is null, the result simply inherits the remaining structure of the other tree.
- **Dominant All `1's` Leaves:** If either `quadTree1` or `quadTree2` is a leaf node with `val=1`, it dominates the OR operation.
- **False Leaves:** If both nodes are leaves but both value is false, then return either tree.

Step 2: Divide and merging

- If the base cases are not met, the current region must be subdivided.
- We continue call `intersect` and calculate on four quadrants (topLeft, topRight, bottomLeft, bottomRight)
- After the recursion returns from the four children, the current node evaluates them.

- If all four children are leaf nodes and they all share the exact same boolean value, the subdivision is redundant. Then we must convert the current internal node into a leaf node and removes the redundant child nodes.

Complexity Analysis

- **Time Complexity:** $\mathcal{O}(n^2 \log n)$ where n is the side length of the grid.
- **Space Complexity:** $\mathcal{O}(n^2)$

Implementation

ideone.com/ucrO2w

4.2 Construct Quad Tree (LeetCode 427)

Problem Description

This problem asks us to construct a Quad-Tree representation from an $n \times n$ binary matrix.
A Quad-Tree is a tree data structure where each internal node has exactly four children representing the four quadrants: top-left, top-right, bottom-left, and bottom-right. Each node in the tree has two properties:

- **val**: A boolean value (True for 1s, False for 0s)
- **isLeaf**: A boolean indicating whether the node is a leaf (True) or has four children (False)

The construction follows these rules:

- **If a region contains all the same values** (all 0s or all 1s):
  - Create a leaf node with `isLeaf = True`
  - Set `val` to the grid's value (True for 1, False for 0)
  - The four children are set to null

- **If a region contains mixed values** (both 0s and 1s):
  - Create an internal node with `isLeaf = False`
  - Set `val` to any value (it doesn't matter since it's not a leaf)
  - Divide the current region into four equal quadrants
  - Recursively construct four children nodes for each quadrant

Solution Approach

This solution uses a divide-and-conquer recursion to construct the Quad-Tree. Here is how the algorithm works step by step.

Step 1: Check if the current region is uniform

```
bool isSame(vector<vector<int>>& grid, int r, int c, int size){
    int val = grid[r][c];
    for(int i = r; i < r + size; i++){
        for(int j = c; j < c + size; j++){
            if(grid[i][j] != val) return false;
        }
    }
    return true;
}
```

- We check whether all cells in the current square region have the same value.
- If the region is uniform, we create a leaf node and stop recursion.

Step 2: Recursively divide the region

```
Node* build(vector<vector<int>>& grid, int r, int c, int size){
    if(isSame(grid, r, c, size)){
        return new Node(grid[r][c], true);
    }

```

```
    int half = size / 2;

    Node* topLeft = build(grid, r, c, half);
    Node* topRight = build(grid, r, c + half, half);
    Node* bottomLeft = build(grid, r + half, c, half);
    Node* bottomRight = build(grid, r + half, c + half, half);

    return new Node(true, false, topLeft, topRight, bottomLeft, bottomRight);
}
```

- If the region contains mixed values, we divide the square into four equal quadrants.
- We recursively construct Quad-Trees for each quadrant.
- The current node becomes an internal node with four children.

Complexity Analysis

- **Time Complexity**: $O(n^2 \log n)$ in the worst case because we may check each region multiple times during recursion.
- **Space Complexity**: $O(n^2)$ for the recursion tree and Quad-Tree nodes.

Implementation

ideone.com/kcYwew

4.3 Retrieve Cell Value in a Quad-Tree

Problem Description

Given a Quad-Tree that represents an $n \times n$ binary matrix and a specific coordinate (r, c), where $0 \leq r, c < n$, the task is to retrieve the value of the cell at that position without reconstructing the entire matrix.

The Quad-Tree uses the same `Node` structure:

- **isLeaf**: True if the node represents a uniform region.
- **val**: The value of the region (if it is a leaf).
- **children**: `topLeft`, `topRight`, `bottomLeft`, `bottomRight` (if it is not a leaf).

Solution Approach

Because a Quad-Tree is a spatial partition, we can use the coordinates (r, c) to decide which quadrant to move into at each step. This is a top-down traversal similar to searching in a Binary Search Tree, but with four branches.

Step 1: Base Case - Leaf Node

If the current node is a leaf (`isLeaf == true`), every cell within this quadrant has the same value. Therefore, the value at (r, c) is simply `node->val`.

Step 2: Recursive Step - Determine the Quadrant

If the node is not a leaf, we calculate the midpoint of the current region ($mid = size/2$) to determine which of the four children contains the target coordinate:

- If $r < mid$ and $c < mid$: Target is in **topLeft**.
- If $r < mid$ and $c \geq mid$: Target is in **topRight** (adjust c to $c - mid$).
- If $r \geq mid$ and $c < mid$: Target is in **bottomLeft** (adjust r to $r - mid$).
- If $r \geq mid$ and $c \geq mid$: Target is in **bottomRight** (adjust r to $r - mid$ and c to $c - mid$).

Complexity Analysis

- **Time Complexity**: $O(\log n)$ or $O(H)$, where H is the height of the tree. In the worst case, we travel from the root to a leaf at the maximum depth.
- **Space Complexity**: $O(\log n)$ due to the recursion stack.

Implementation

ideone.com/5djGhW

4.4 Counting Necessary Nodes (Codeforces 2074F)

Problem Description

A **quadtree** is a tree data structure in which each node has at most four children and accounts for a square-shaped region.

Formally, **for all tuples** of nonnegative integers $k, a, b \geq 0$, there exists **exactly one node** accounting for the following region:

$$[a \cdot 2^k, (a+1) \cdot 2^k] \times [b \cdot 2^k, (b+1) \cdot 2^k]$$

All nodes whose region is larger than 1×1 contain four children corresponding to the regions divided equally into four, and the nodes whose region is 1×1 correspond to the leaf nodes of the tree.

The pink soldiers have given you a finite region $[l_1, r_1] \times [l_2, r_2]$, where l_i and r_i ($l_i < r_i$) are nonnegative integers. Please find the minimum number of nodes that you must choose in order to make the union of regions accounted for by the chosen nodes **exactly** the same as the given region. Here, two sets of points are considered different if there exists a point included in one but not in the other.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The only line of each test case contains four integers l_1, r_1, l_2, r_2 — the boundaries of the region in each axis ($0 \leq l_i < r_i \leq 10^6$).

Output

For each test case, output the minimum number of nodes necessary to satisfy the condition on a separate line.

Example

Input:

```
5
0 1 1 2
0 2 0 2
1 3 1 3
0 2 1 5
9 98 244 353
```

Output:

```
1
1
4
5
374
```

Solution Approach

Notice how the intervals on each axis correspond to those found in nodes of standard 1D segment trees. Let us find the segment tree intervals required to cover the query range on each axis individually, and call those sets of intervals I_x and I_y.

Then, for each rectangular region formed by some $a \times b$ such that $a \in I_x$ and $b \in I_y$, the following properties hold:

1. No quadtree node with side length strictly greater than $\min(|a|, |b|)$ can cover this rectangle without spilling outside the given total region.

2. All quadtree nodes with side length no greater than $\min(|a|, |b|)$ are either completely inside the rectangle or completely outside.

Thus, we find the following strategy to cover the region optimally: For all rectangles $a \times b$ such that $a \in I_x$ and $b \in I_y$, cover the sub-region using the largest possible valid nodes. Since both $|a|$ and $|b|$ are powers of two (as they are segment tree nodes), the larger side length is perfectly divisible by the smaller side length.

Therefore, we can completely tile the $a \times b$ sub-region with square nodes of side length $\min(|a|, |b|)$. The number of square nodes required to cover this specific sub-region is:

$$\frac{\max(|a|, |b|)}{\min(|a|, |b|)}$$

Summing this value over all pairs of $a \in I_x$ and $b \in I_y$ gives the final answer. Implementing this leads to a solution with time complexity $\mathcal{O}(\log^2 X)$, where X is the maximum coordinate value. While faster $\mathcal{O}(\log X)$ solutions exist, they are not strictly necessary to pass the time limits for this problem.

Implementation

ideone.com/aV7CoR

5 Appendices

- Reference: `https://hypersphere.blog/blog/quad-trees/`
- Github Link: `https://github.com/khanh47/QuadTree`
- Visualizer Link:

  `https://opendsa-server.cs.vt.edu/ODSA/Books/CS3/html/PRquadtree.html`